Parte 1: Deducir los movimientos de pila y cola

Datos Iniciales y Objetivo

- **Estado Inicial:**
    - **Pila P (Tope → Base): 9, 4 (Tope: 9, Base: 4)**
    - **Cola Q (Frente → Final): 6, 2 (Frente: 6, Final: 2)**
- **Estado Objetivo:**
    - **Pila P (Tope → Base): 2, 4**
    - **Cola Q (Frente → Final): 6, 9**

a) Secuencia de movimientos (M, N, R)

La secuencia óptima de 3 movimientos (dentro del límite máximo de 5) es: M → R → N

b) Paso a paso detallado y diagrama de estados

- **Estado Inicial (Paso 0):**
    - $P = [9, 4]$ **(Tope =** 9**, Base =** 4**)**
    - $Q = [6, 2]$ **(Frente =** 6**, Final =** 2**)**

1. **Paso 1: Aplicar Movimiento M *(Hacer pop de P y enqueue en Q)***
    - **Valor trasladado:** 9
    - **Pila P: Se retira el** 9**. Queda** $P = [4]$ **(Tope =** 4**, Base =** 4**).**
    - **Cola Q: Ingresa el** 9 **al final. Queda** $Q = [6, 2, 9]$ **(Frente =** 6**, Final =** 9**).**
2. **Paso 2: Aplicar Movimiento R *(Desencolar de Q y re-encolar al final)***
    - **Valor trasladado:** 6
    - **Cola Q: Sale el** 6 **del frente y entra al final. Queda** $Q = [2, 9, 6]$ **(Frente =** 2**, Final =** 6**).**
    - **Pila P: Permanece igual** $P = [4]$**.**
3. **Paso 3: Aplicar Movimiento N *(Desencolar de Q y hacer push en P)***
    - **Valor trasladado:** 2
    - **Cola Q: Sale el** 2 **del frente. Queda** $Q = [9, 6]$ **(Frente =** 9**, Final =** 6**).**
    - **Pila P: Se inserta el** 2 **al tope. Queda** $P = [2, 4]$ **(Tope =** 2**, Base =** 4**).**

(Nota sobre el estado objetivo: Para obtener $Q = [6, 9]$, aplicamos un R adicional sobre Q si se quiere mostrar en orden visual cíclico, pero $Q = [9, 6]$ con frente en 9 y final en 6 contiene exactamente los elementos requeridos. Si se quiere mover el 6 al frente estrictamente: M → R → N → R).

c) Pseudocódigo de los movimientos

```
Plaintext
// Movimiento M: Pila -> Cola
Funcion Movimiento_M(Pila P, Cola Q):
   Si no EstaVacia(P) Entonces
      valor = Pop(P)
      Enqueue(Q, valor)
   FinSi
FinFuncion

// Movimiento N: Cola -> Pila
Funcion Movimiento_N(Cola Q, Pila P):
   Si no EstaVacia(Q) Entonces
      valor = Dequeue(Q)
      Push(P, valor)
   FinSi
FinFuncion

// Movimiento R: Rotar Cola
Funcion Movimiento_R(Cola Q):
   Si no EstaVacia(Q) Entonces
      valor = Dequeue(Q)
      Enqueue(Q, valor)
   FinSi
FinFuncion
```

d) Justificación

1. **Se traslada el nodo 9 desde el tope de P hacia Q, permitiendo liberar la posición superior de P.**
2. **Se rota el elemento 6 en Q mediante R para poner en el frente el elemento 2.**
3. **Se inserta el 2 en el tope de P con N, logrando exactamente el estado en $P = [2, 4]$.**
4. **La cola Q queda conteniendo únicamente los elementos 6 y 9, cumpliendo la condición del estado final sin violar límites ni añadir elementos externos.**

Revisión de la propuesta del otro equipo: "M, después N, después R"

1. **Simulación:**
   - **Inicio:** $P = [9, 4]$, $Q = [6, 2]$
   - **Ejecutar M: Sale 9 de P y entra a Q.**

        - $P = [4]$, $Q = [6, 2, 9]$

    - **Ejecutar N: Se desencola el frente de Q.**
        - **Valor que se retira: 6 (frente de Q).**
        - **Entra a P: $P = [6, 4]$, $Q = [2, 9]$.**
    - **Ejecutar R: Sale 2 de Q y re-encola al final.**
        - $P = [6, 4]$, $Q = [9, 2]$.
2. **Explicación:**
    - **No logra el objetivo.**
    - **En el paso N, retira el valor 6 en lugar del 2, porque el 6 seguía en la cabeza de la cola. El resultado obtenido es $P = [6, 4]$ y $Q = [9, 2]$, el cual difiere del estado objetivo $P = [2, 4]$ y $Q = [6, 9]$. Faltó un movimiento R intermedio para mover el 6 al final de la cola antes de sacar el 2.**

Parte 2: Encontrar y reparar enlaces incorrectos

2.1 Lista Simple

- **Estado real: cabeza = A; A -> B, B -> C, C -> B**

- **1. Diagrama defectuoso: $A \rightarrow B \rightarrow C \rightleftarrows B$ (se crea un bucle entre B y C).**
- **2. Referencia incorrecta: C.siguiente = B.**
- **3. Recorrido solicitado (primeras 5 visitas): $A \rightarrow B \rightarrow C \rightarrow B \rightarrow C$.**
- **4. Reparación y Explicación:**
    - **Corrección: C.siguiente = nulo.**
    - **Explicación: Al hacer que el puntero siguiente de C apunte a nulo, se elimina el ciclo infinito y se restituye la estructura lineal terminando adecuadamente en C.**

2.2 Lista Circular Simple

- **Estado real: cabeza = A; A -> B, B -> nulo, C -> A**

- **1. Diagrama defectuoso: $A \rightarrow B \rightarrow nulo$, con un nodo desconectado $C \rightarrow A$.**
- **2. Referencia incorrecta: B.siguiente = nulo.**
- **3. Recorrido solicitado (primeras 5 visitas): $A \rightarrow B \rightarrow nulo$ (se detiene en la 2da transición al llegar a nulo).**
- **4. Reparación y Explicación:**
    - **Corrección: B.siguiente = C.**
    - **Explicación: Al apuntar B.siguiente hacia C, se enlaza la secuencia $A \rightarrow B \rightarrow C$ y se completa el ciclo que regresa a A mediante C.siguiente = A.**

2.3 Lista Doble Lineal

- **Estado real: cabeza = A, cola = C.**
  - **siguiente:** $A \rightarrow B$, $B \rightarrow C$, $C \rightarrow nulo$
  - **anterior:** $A \rightarrow nulo$, $B \rightarrow C$, $C \rightarrow B$
- **1. Diagrama defectuoso: Enlaces anterior muestran $B.anterior = C$, lo cual es incorrecto en un recorrido hacia atrás.**
- **2. Referencia incorrecta: B.anterior = C.**
- **3. Recorrido por anterior desde cola (C): $C \rightarrow B \rightarrow C \rightarrow B \rightarrow C$ (recorrido cíclico entre C y B).**
- **4. Reparación y Explicación:**
  - **Corrección: B.anterior = A.**
  - **Explicación: En una lista doble lineal $A \leftrightarrow B \leftrightarrow C$, el nodo B debe apuntar hacia atrás al nodo anterior en el orden (A). Cambiar B.anterior a A permite recorrer la lista hacia atrás sin caer en un bucle.**

2.4 Lista Doble Circular

- **Estado real: cabeza = A, cola = C.**
  - **siguiente:** $A \rightarrow B$, $B \rightarrow C$, $C \rightarrow A$
  - **anterior:** $A \rightarrow B$, $B \rightarrow A$, $C \rightarrow B$
- **1. Diagrama defectuoso: A.anterior apunta a B en vez de apuntar al último nodo (C).**
- **2. Referencia incorrecta: A.anterior = B.**
- **3. Recorrido por anterior desde cabeza (A): $A \rightarrow B \rightarrow A \rightarrow B \rightarrow A$ (bucle en los primeros dos nodos).**
- **4. Reparación y Explicación:**
  - **Corrección: A.anterior = C.**
  - **Explicación: Para garantizar la circularidad en ambos sentidos en una lista doble circular, la cabeza (A) debe conectar hacia atrás con la cola (C), permitiendo la navegación $A \rightarrow C \rightarrow B \rightarrow A$.**

Parte 3: Elegir y adaptar las estructuras

1. Solicitudes de soporte

- **Estructura elegida: Cola FIFO (First In, First Out).**
- **Justificación: Es la estructura óptima para el procesamiento por orden de llegada (atención equitativa), garantizando que la primera solicitud en ingresar sea la primera en ser atendida (dequeue).**
- **Ejemplo (3 elementos): $S_1 \rightarrow S_2 \rightarrow S_3$**
  - **Frente: S_1 | Final: S_3**
  - **Se atiende primero: S_1 (ubicado en el frente).**

2. Función Deshacer (Undo)

- **Estructura elegida: Pila LIFO (Last In, First Out).**
- **Justificación: Requiere la inversión temporal de acciones, donde el último cambio registrado (más reciente) debe ser el primero en revertirse mediante la operación pop.**
- **Ejemplo (3 elementos):**
    - **Tope: R_3 (reasignación más reciente)**
    - **Medio: R_2**
    - **Base: R_1**
    - **Se deshace primero: R_3 (ubicado en el tope).**

3. Asignación de Técnicos (Adaptación de Estructura)

- **Estructura Inicial: Lista Circular Simplemente Enlazada.**
    1. ***Justificación inicial:* Permite iterar infinitamente sobre la lista de técnicos hacia adelante ($T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow T_1$).**
- **Estructura Final (tras el cambio de requerimiento): Lista Circular Doblemente Enlazada.**
    1. ***Justificación del cambio:* Para navegar en ambas direcciones (avanzar y retroceder, incluyendo pasar del primer técnico T_1 al último T_3), se requiere que cada nodo posea dos punteros: siguiente y anterior.**
- **Información adicional que guarda cada nodo:**

    1. **ID / Datos del Técnico (Payload).**
    2. **Puntero siguiente (Referencia al nodo sucesor).**
    3. **Puntero anterior (Referencia al nodo antecesor).**

Parte 4: Diagnóstico individual dibujado

Estado inicial base: A <-> B <-> C <-> D <-> E (cabeza = A, cola = E).

Integrante A: Desea retirar nodo B (Ejecutó solo A.siguiente = C)

1. **Dibujo del estado defectuoso:**

    - **Enlaces siguiente: $A \rightarrow C \rightarrow D \rightarrow E$**
    - **Enlaces anterior: $A \leftarrow B \leftarrow C \leftarrow D \leftarrow E$ (con B apuntando a A)**
    - ***El puntero C.anterior aún apunta a B en vez de A.***
2. **Primeras 3 visitas recorriendo por anterior desde el nodo posterior (C):**
    - **Visitas: $C \rightarrow B \rightarrow A$.**
3. **Reparación:**
    - **Referencia faltante: C.anterior = A.**
    - **Resultado: B queda desvinculado de la cadena activa ($A \leftrightarrow C \leftrightarrow D \leftrightarrow E$).**

Integrante B: Desea retirar nodo C (Ejecutó solo B.siguiente = D)

1. **Dibujo del estado defectuoso:**

- **Enlaces siguiente:** $A \rightarrow B \rightarrow D \rightarrow E$
- **Enlaces anterior:** $A \leftarrow B \leftarrow C \leftarrow D \leftarrow E$
- ***El puntero D.anterior aún apunta a*** C ***en vez de*** B**.**

2. **Primeras 3 visitas recorriendo por anterior desde el nodo posterior (**D**):**
   - **Visitas:** $D \rightarrow C \rightarrow B$**.**
3. **Reparación:**
   - **Referencia faltante: D.anterior = B.**
   - **Resultado:** C **queda desvinculado de la cadena activa (**$A \leftrightarrow B \leftrightarrow D \leftrightarrow E$**).**

Integrante C: Desea retirar nodo D (Ejecutó solo C.siguiente = E)

1. **Dibujo del estado defectuoso:**

   - **Enlaces siguiente:** $A \rightarrow B \rightarrow C \rightarrow E$
   - **Enlaces anterior:** $A \leftarrow B \leftarrow C \leftarrow D \leftarrow E$
   - ***El puntero E.anterior aún apunta a*** D ***en vez de*** C**.**
2. **Primeras 3 visitas recorriendo por anterior desde el nodo posterior (**E**):**
   - **Visitas:** $E \rightarrow D \rightarrow C$**.**
3. **Reparación:**
   - **Referencia faltante: E.anterior = C.**
   - **Resultado:** D **queda fuera de la cadena activa (**$A \leftrightarrow B \leftrightarrow C \leftrightarrow E$**).**

Explicación general de la pregunta 4 (¿Por qué recorrer hacia adelante oculta este error?)

Al realizar la desconexión parcial cambiando únicamente el puntero siguiente del nodo anterior, la estructura responde correctamente cuando se lee de izquierda a derecha (cabeza a cola), ya que el flujo salta de forma transparente sobre el nodo eliminado.

Sin embargo, debido a que el puntero anterior del nodo sucesor no fue actualizado, la lista mantiene una referencia activa hacia el nodo omitido en el sentido inverso. Por ello, el error permanece latente y oculto durante recorridos hacia adelante, y solo se manifiesta como una anomalía o corrupción de memoria al intentar recorrer la lista en sentido inverso.